Murmur8 OS: Product Vision

Oct 17, 2026 · @Steve

An operating system built for one job: a developer working alongside many AI agents, entirely from the terminal.

Vision

A clean and calm working environment for Agentic software development. The OS is lean, efficient and is optimised, specifically, for the hardware it has available. It boots straight into an environment where every agent and shell is a first-class, instantly switchable session. Agents from different harnesses coordinate through a small, built-in pub/sub message bus. This 'message board' is on the minimalistic desktop so a user can watch messages in real time whilst working with agents in dfferent terminals. It is a customised Linux distribution built on Arch: a stable base tuned to a 2016 MacBook Pro. Keys to things like GitHub or Azure are safe in a Vault, which the agents can ask to be forwarded to a respective service, but can't be accessed directly.

Who it's for

Its author comes first: a web developer with a C# and Java background who already works with agents in terminals and wants an environment shaped around that way of working. The code lives in a public repo, so anyone who wants a hackable, hardware-specific dev machine can learn from it or fork it, but nothing is designed for a wide audience.

Principles

1. **Terminal-first.** Keyboard-driven, fast switching between sessions, minimal chrome.
2. **Agents as citizens.** A built-in pub/sub bus lets agents and tools from any harness publish, subscribe and coordinate.
3. **Built for the hardware.** Every machine gets a tuned profile rather than a lowest-common-denominator install.
4. **Modular by design.** A shared core plus swappable hardware profiles, so a new machine is a new profile, not a fork.
5. **Stable base, fresh tools.** The hardware layer is pinned and advanced on purpose; dev tools and agents update at their own pace.

6. **Understandable.** Small enough to read and learn from. The learning happens in kernel configuration, drivers, boot, packaging and the agent bus.
7. **Beat obsolescence.** Give capable old hardware a long second life.

Non-goals

- A general consumer desktop.
- Broad hardware support.
- Replacing macOS or Linux for everyone.
- Running on hardware nobody has profiled.
- A from-scratch kernel on the main path. It stays an optional side project.

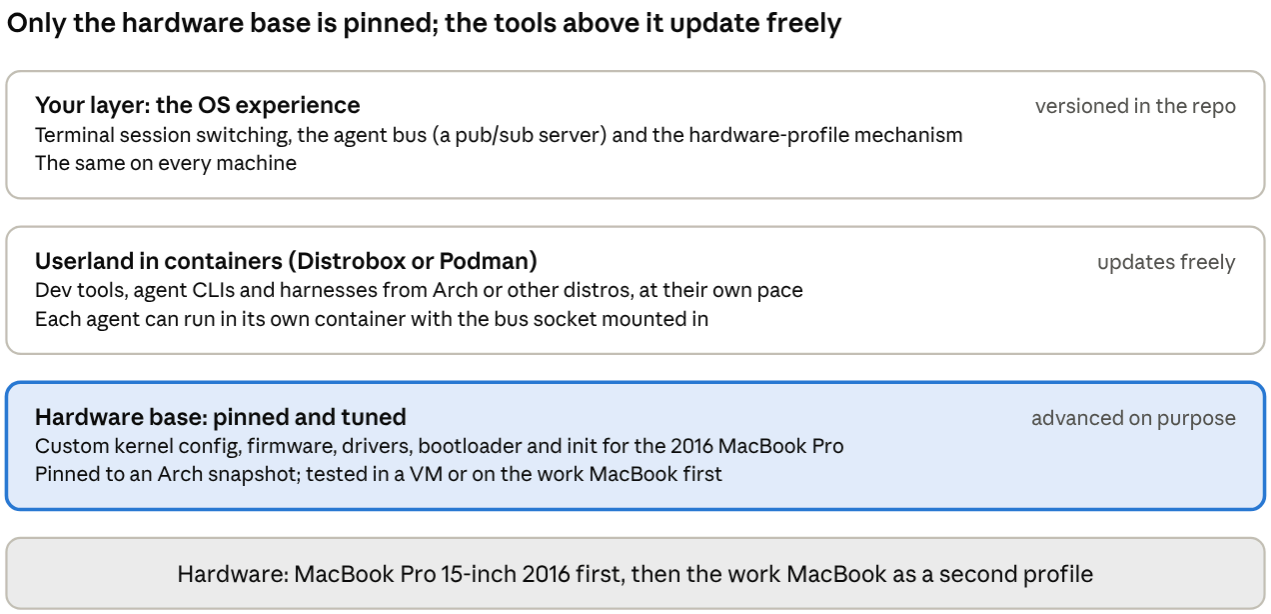
Target hardware

The first target is a MacBook Pro (15-inch, 2016): 2.7 GHz quad-core Intel Core i7, 16 GB 2133 MHz LPDDR3, Intel HD Graphics 530 with 1536 MB. Intel publishes detailed documentation for its graphics, chipset and CPU, which makes this a friendlier learning target than Apple Silicon. A second, older work MacBook is the planned test of the modular profile idea.

Area	Reality on this generation	Early plan
Graphics	Intel HD 530. Most 15-inch 2016 models also carry a discrete AMD Radeon Pro (to verify in About This Mac)	Intel only
Keyboard and trackpad	Connected over SPI, not USB	External USB keyboard during bring-up
Wi-Fi	Broadcom, needs proprietary firmware	Linux's Broadcom support; USB ethernet as a fallback
Audio, Touch Bar, suspend/resume	Usually need out-of-tree Linux drivers	Defer
Debugging	No serial port	Test images in QEMU or a VM first; logs from the real machine over USB-C ethernet

Approach: a stable hardware base with fresh tools above it

Build on Arch Linux instead of a new kernel: pin and tune the hardware layer, and keep dev tools and agents in containers that update freely.



layers · 1 pinned base under 2 free-moving layers

The base is pinned to a dated Arch snapshot and advanced on purpose, so breakage only arrives when you choose to update. Containers hold the fast-moving tools, and each agent can run in its own container with the bus socket mounted in. A from-scratch Rust kernel in QEMU stays available as an optional side project.

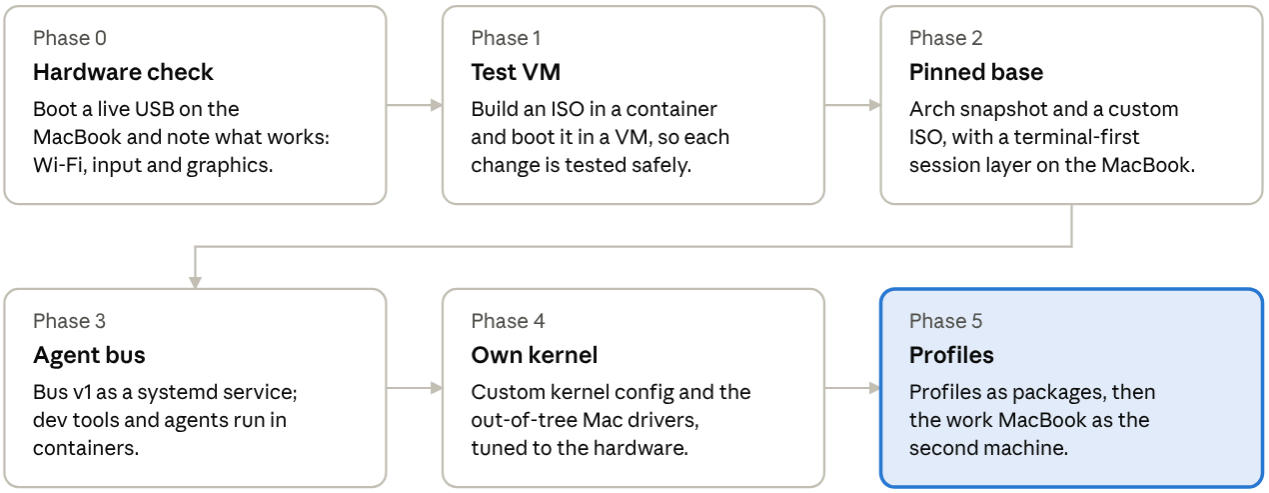
Keeping the base stable

- Pin the base to a dated snapshot from the Arch Linux Archive, and advance it every few months.
- Test each new snapshot in a VM or on the work MacBook before promoting it to the main machine.
- Keep your own mirror of the packages you depend on, so the OS can always be rebuilt.
- Take a Btrfs snapshot before every upgrade, so you can roll back.
- Plan for security patches: a pinned base still needs them, and browser, OpenSSL and kernel fixes matter most.

Roadmap

Six phases, in order, take the project from a live USB test on the MacBook to a second MacBook running from a profile.

Six phases lead to the test: a second MacBook from a profile, not a rewrite



Phases run in order and are not drawn to scale. A from-scratch Rust kernel in QEMU stays an optional side project.

roadmap · 6 phases, 1 test

Phase 1 gives every later phase a safe, fast test loop. Phases 2 to 4 build the stack from the bottom up. Phase 5 is the test of the modular profile idea.

Success looks like

- You work a full day on it.
- The agent bus is something you would miss on any other machine.
- A second MacBook comes up from a hardware profile rather than a rewrite.